Case Study 1: University Scholarship Allocation

Objective: To check eligibility for scholarship.
Assumption: Marks ≥ 75, Attendance ≥ 75, Income ≤ 3L.
Problem Statement: Check eligibility using if-else condition.
Algorithm:
Step 1: Start
Step 2: Input marks, attendance, income
Step 3: Check if marks ≥ 75 and attendance ≥ 75 and income ≤ 300000
Step 4: Print Eligible else Not Eligible
Step 5: End.
Pseudocode:

```
READ marks, attendance, income
IF marks ≥ 75 AND attendance ≥ 75 AND income ≤ 300000 THEN
PRINT Eligible
ELSE
PRINT Not Eligible
END
```

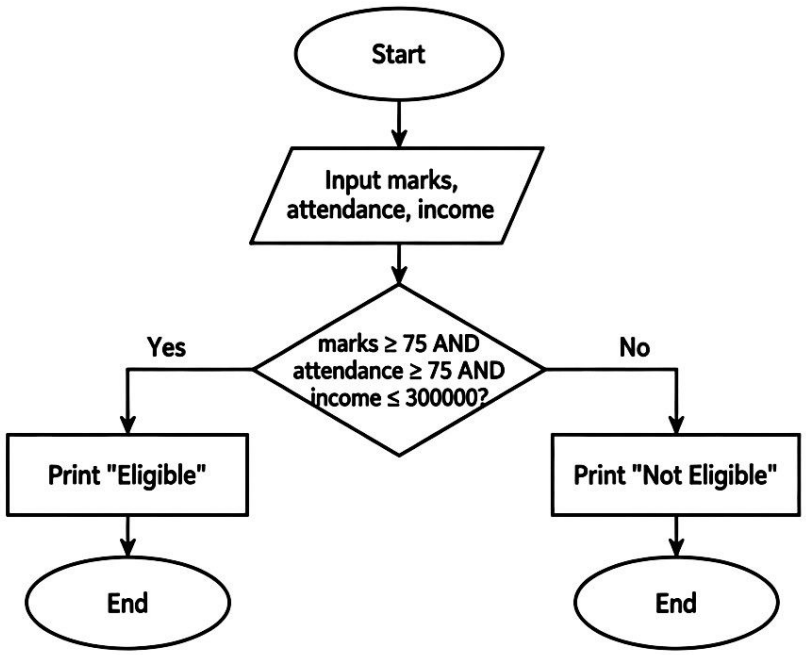
Source Code:

```
marks = int(input("Aggregate: ")
att = int(input("Attendance: ")
else:    = print("Not Eligible"
```

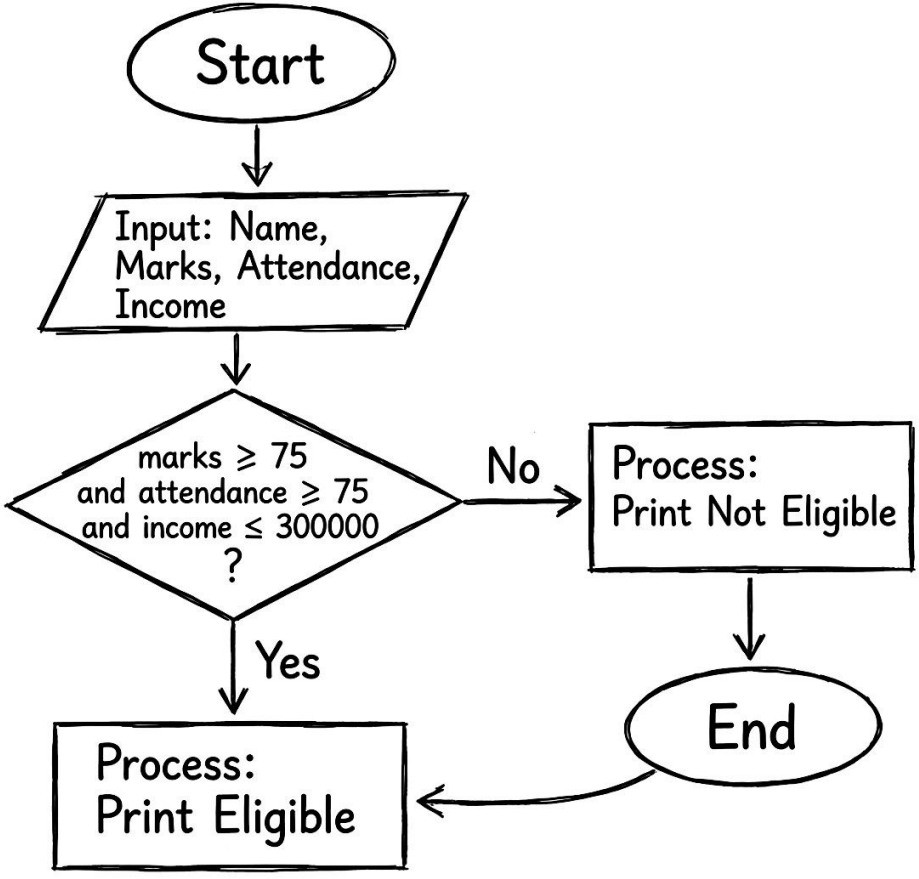
Test Cases:
Test Case 1: marks = 80, att = 80, income = 200000 → Output: Eligible
Test Case 2: marks = 60, att = 90, income = 200000 → Output: Not Eligible
Test Case 3: marks = 80, att = 60, income = 200000 → Output: Not Eligible

Output: As shown in Test Cases above.

Flowchart: See below (Oval = Start/End, Parallelogram = Input/Output, Diamond = Decision, Rectangle = Process)



Scholarship Eligibility Check



Case Study 2: Progressive Electricity Billing

Objective: To calculate electricity bill based on slabs.

Assumption: First 100 units Rs 3 per unit, next units Rs 5 per unit, Fixed charge Rs 50.

Problem Statement: Calculate bill using if-else.

Step 1: Start
Step 2: Input units
Step 3: Check if units <= 100 then **bill = units * 3** else
bill = 300 + **(units - 100) * 5**
Step 4: **bill = bill + 50**
Step 5: Print bill
Step 6: End.

Pseudocode:

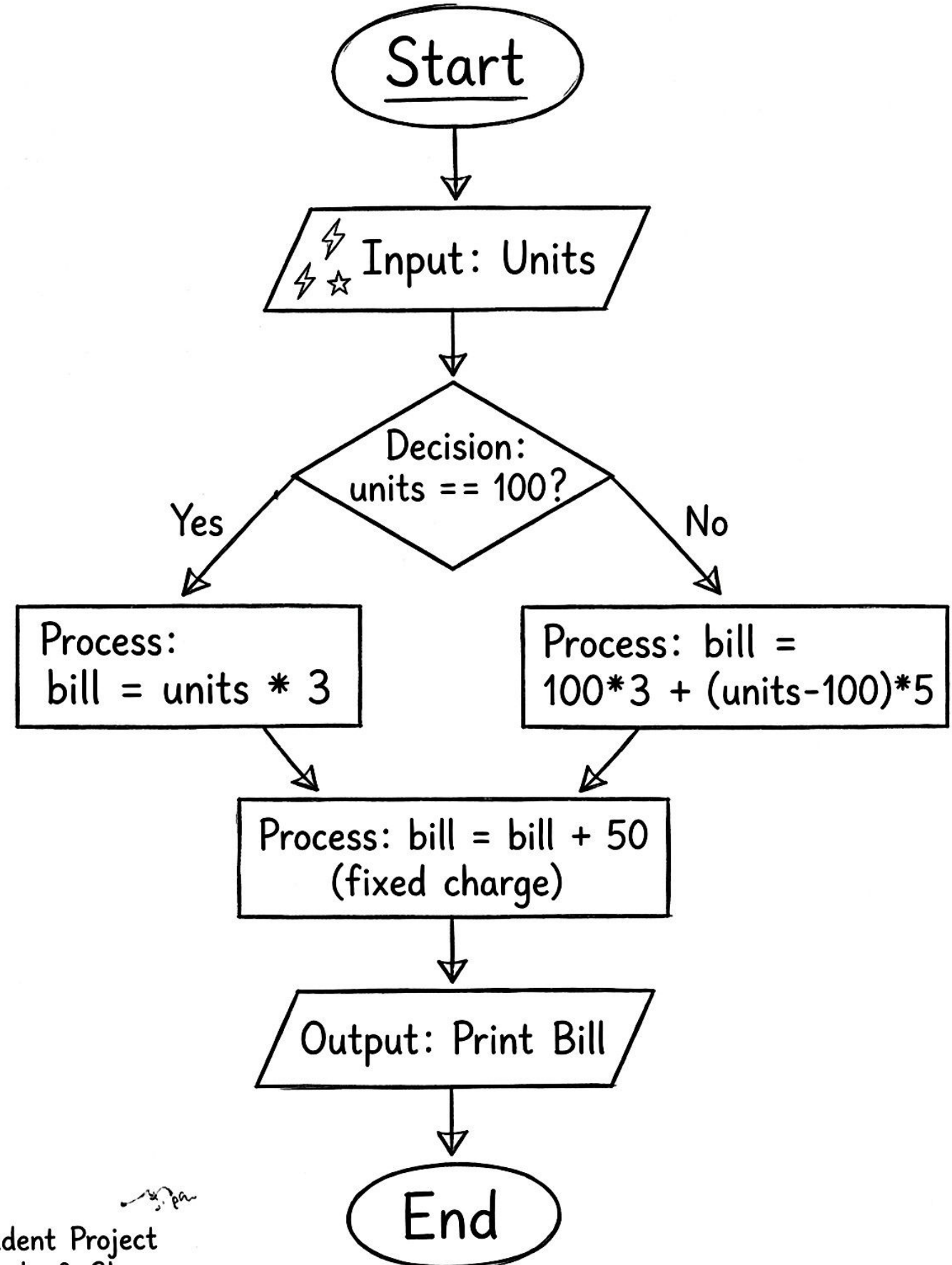
```
READ units
IF units <= 100 THEN
bill = units * 3
ELSE
bill = 300 + (units - 100) * 5
bill = bill + 50
print("Bill:;" bill)
END
```

Test Cases: = int(input("Enter units:))

Test Case 1: units = 80 Output 290
Test Case 2: units = 150 Output 600
Test Case 3: units = 200 Output 855

Output: As shown in Test Cases above.

Electricity Billing Flowchart



- Student Project
• Simple & Clear •

Case Study 3: Hospital Billing and Patient Classification

Case Study 3: Hospital Billing and Patient Classification

Objective: To calculate total hospital bill.

Assumption: Consultation, diagnostic, treatment charges.

Problem Statement: Calculate total and classify.

Algorithm:
Step1: Start
Step2: Input consultation, diagnostic, treatment
Step3: total = consultation + diagnostic + treatment
Step4: Print total
Step5: End.

Pseudocode:

```
READ consultation, diagnostic, treatment
total = consultation + diagnostic + treatment
PRINT total
END
```

Source Code:

```
consultation = int(input("Consult: ")
diagnostic = int(input("Diagnostic: ")
treatment = int(input("Treatment: ")
total = consultation + diagnostic + treatment
print("Total Bill:;" total)
```

Test Cases:
Test Case 1: consult = 500, diag = 500, treat = 1000 → 2000

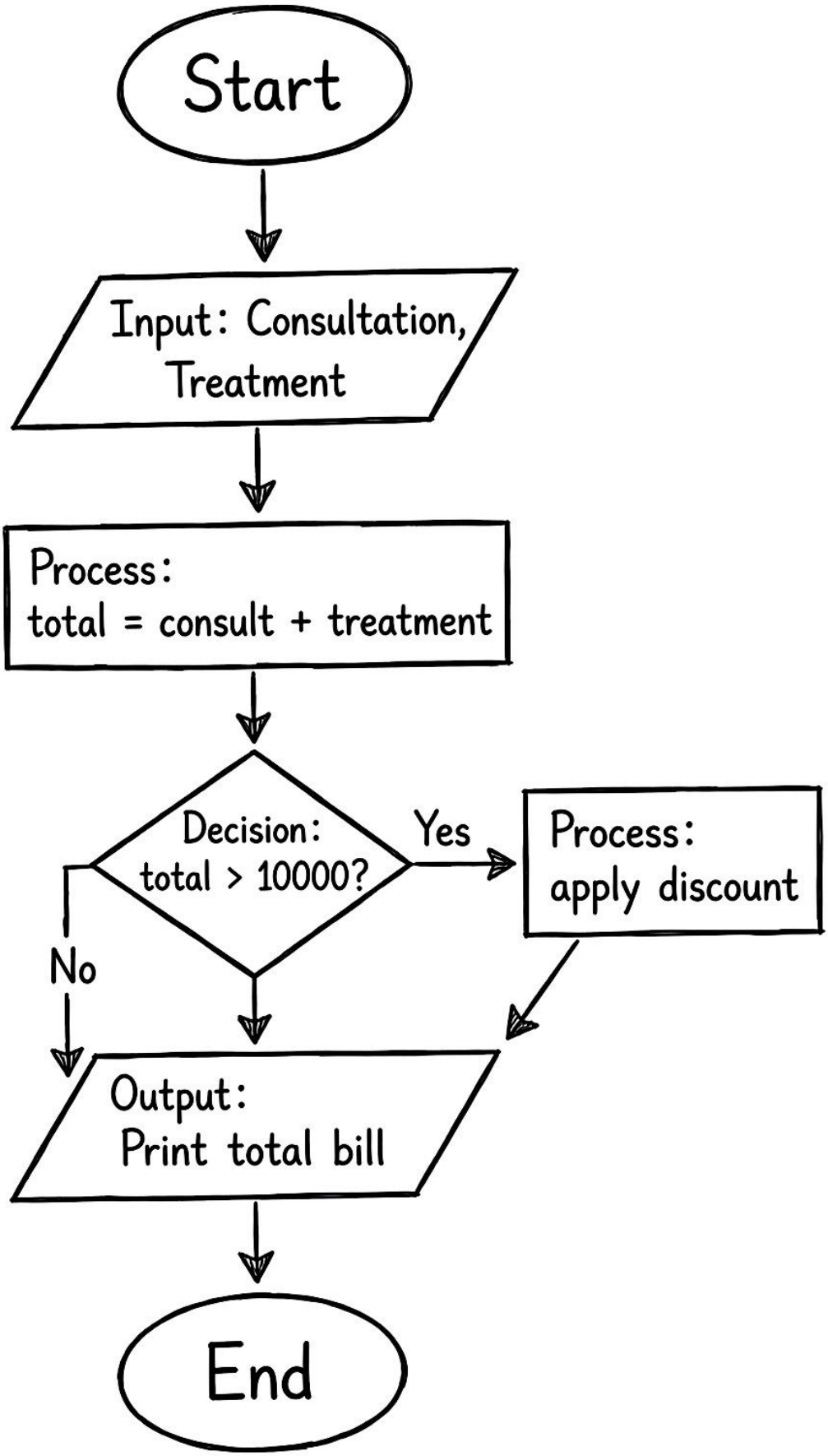
Test Case 2: 1000 + 2000 + 3000 → 6000

Output: As shown above.

Flowchart: See next page.

Hospital Billing Flowchart

Student Project



Case Study 4: Employee Payroll and Deduction Analysis

Objective: To calculate net salary with deductions.

Assumption: Gross = basic + allowances
- Deduction = 10% if gross > 50000, else 5%.

Problem Statement: Calculate net salary.

Algorithm:

Step 1: Start
Step 2: Input basic, allowances
Step 3: gross = basic + allowances
Step 4: If gross > 50000, deduction = 10% of gross, else deduction = 5% of gross
Step 5: net = gross – deduction
Step 6: Print net
Step 7: End.

Pseudocode:

```
BEGIN
    READ basic, allowances
    gross = basic + allowances
    IF gross > 50000 THEN
       deduction = gross * 10 / 100
    ELSE
       deduction = gross * 5 / 100
       net = gross – deduction
    PRINT net
    END.
```

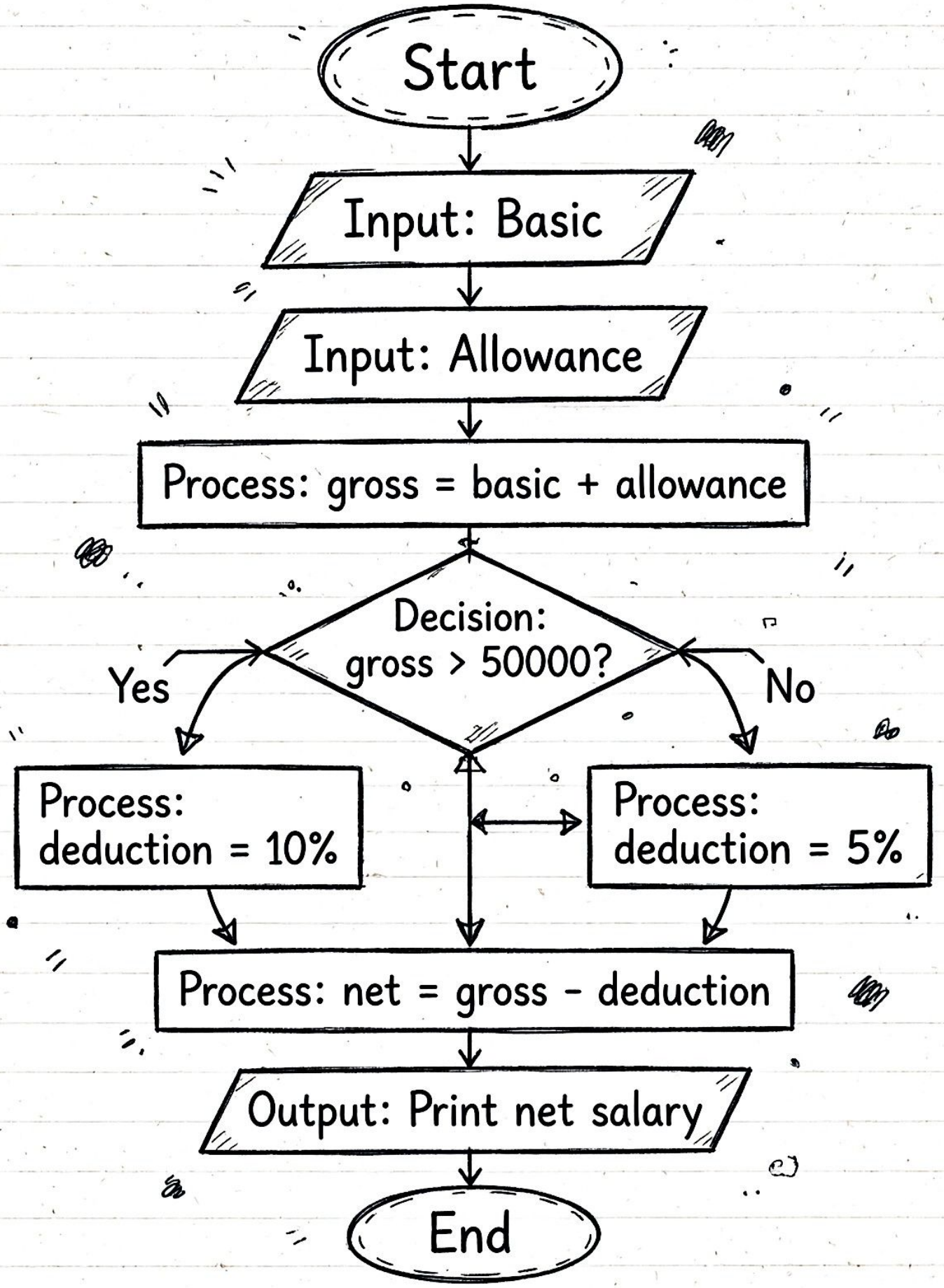
Source Code:

```
basic = int(input("Basic: "))
allow = int(input("Allowance: "))
gross = basic + allow
if gross > 50000:
  ded = gross * 10 // 100
  else:
  ded = gross * 5 // 100
  net = gross - ded
  print("Net Salary:",net)
```

Test Cases:

- basic = 40000, allow = 5000 → net = 42750
- basic = 60000, allow = 5000 → net = 58500

Employee Payroll Flowchart



Case Study 5: Multi-Plan Telecommunications Cost Optimization

Objective: To select economical telecom plan.

Assumption:

- Plan A: usage ≤ 100 → cost = 50
- Plan B: usage ≤ 200 → cost = 80
- Plan C: usage > 200 → cost = 100

Problem Statement: Select the best plan based on the usage input.

Algorithm:

Step 1: Start
Step 2: Input usage
Step 3: If usage ≤ 100 → Print "Plan A, Cost = 50"
elif usage ≤ 200 → Print "Plan B, Cost = 80"
else → Print "Plan C, Cost = 100"
Step 4: End

Pseudocode:

```
BEGINREAD usage
-     IF usage ≤ 100 THEN
      PRINT "Plan A, Cost = 50"
      ELSE IF usage ≤ 200 THEN
-     PRINT "Plan B, Cost = 80"
-    PRINT "Plan C, Cost = 100"
END
```

Source Code (Python):

```
usage = int(input("Usage: "))
if usage <= 100:
   print("Plan A, Cost = 50")
   elif usage <= 200:
   print("Plan B, Cost = 80")
   print("Plan C, Cost = 100")
else:
```

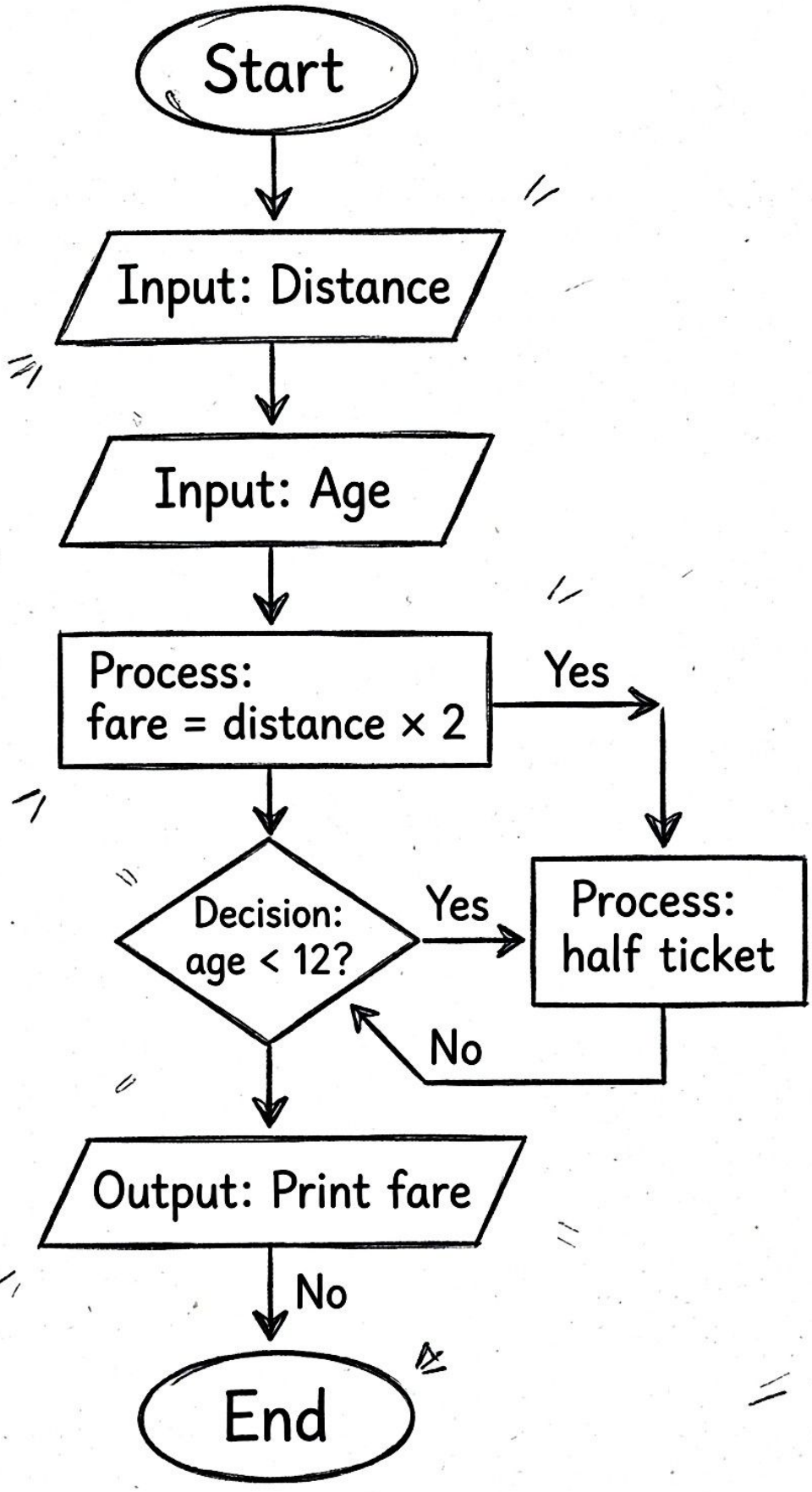
Test Cases:

- Input: usage = 80 → Output: Plan A, Cost = 50
- Input: usage = 150 → Output: Plan B, Cost = 80
- Input: usage = 250 → Output: Plan C, Cost = 100

Output: As shown above in the test cases.

Flowchart: See next page.

Railway Fare Calculation



Case Study 6

Railway Fare and Reservation Calculator

Objective: To calculate railway fare based on distance and age.

Assumption:

- fare = distance × 2
- Half fare if age < 12

Problem Statement: Calculate fare with concession.

Algorithm:

Step 1: Start
Step 2: Input distance
fare = distance × **2**
Step 3: Input age
Step 4: If age < **12** fare = fare / **2**
Step 5: Print fare
Step 6: End.

Pseudocode:

```
BEGIN
READ distance
fare = distance × 2
READ age
IF age < 12 THEN
fare = fare / 2
PRINT fare
END.
```

Source Code (Python):

```
distance = int(input("Distance:"))
fare = distance * 2
age = int(input("Age:"))
if age < 12:
·  print("Half", fare // 2)
·  else:
·  print("Full", fare)
```

Test Cases:

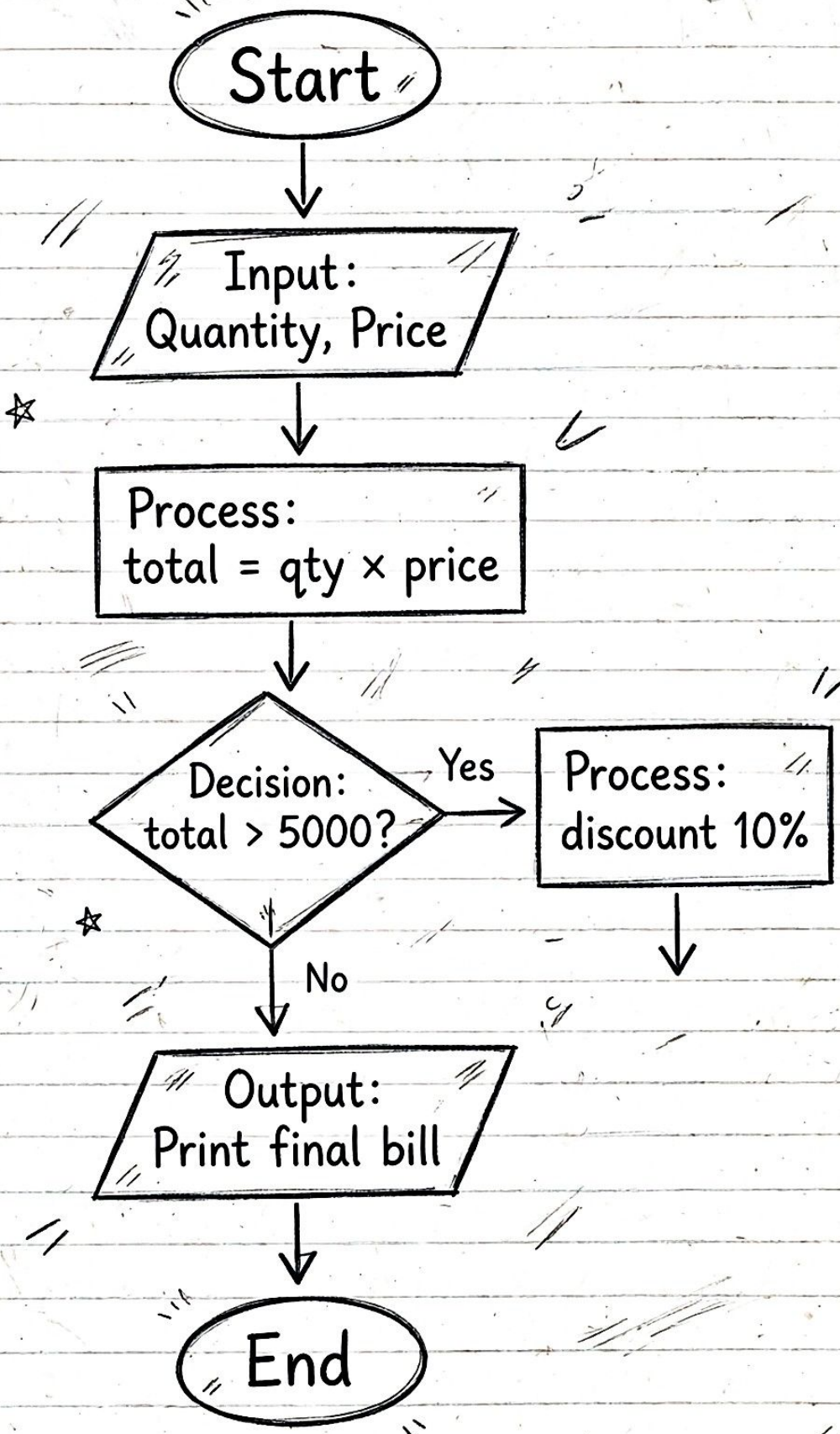
Distance	Age	Output
100	25	Full 200
100	10	Half 100
500	8	Half 500

Output: As shown above.

Flowchart: See next page.

— End of Case Study 6 —

Retail Billing – Flowchart



Case Study 7: Retail Billing with Multiple Discount Rules

Objective: To calculate retail bill with discount and tax.

Assumption: total = quantity * price
discount 10% if total > 5000
GST 18% if electronics.

Problem Statement:
Generate itemized invoice.

Algorithm:
Step 1: Start
Step 2: Input quantity, price
Step 3: total = quantity * price
Step 4: If total > 5000, discount 10%
Step 5: Add GST 18% if applicable (electronics)
Step 6: Print final bill
Step 7: End.

Pseudocode:

```
BEGIN
READ quantity, price
total = quantity * price
IF total > 5000 THEN
·   total = total - (total * 10 / 100)
  IF item_type = "electronics" THEN
·   total = total + (total * 18 / 100)
PRINT total
END.
```

Source Code:

```
quantity = int(input("Enter Quantity: "))
price = int(input("Enter Price per Item: "))
item_type = input("Is item electronic? (yes/no): ").lower()
total = quantity * price
discount = 0
if total > 5000:
·   discount = total * 0.10
·   total = total - discount
·   gst = 0
if item_type == "yes" or item_type == "electronics"::
·   gst = total * 0.18
print("Subtotal:;" quantity, *:;" price, =;," quantity*price)
print("GST::" gst)
print("Final Bill::" round(total, 2))
```

Test Cases:

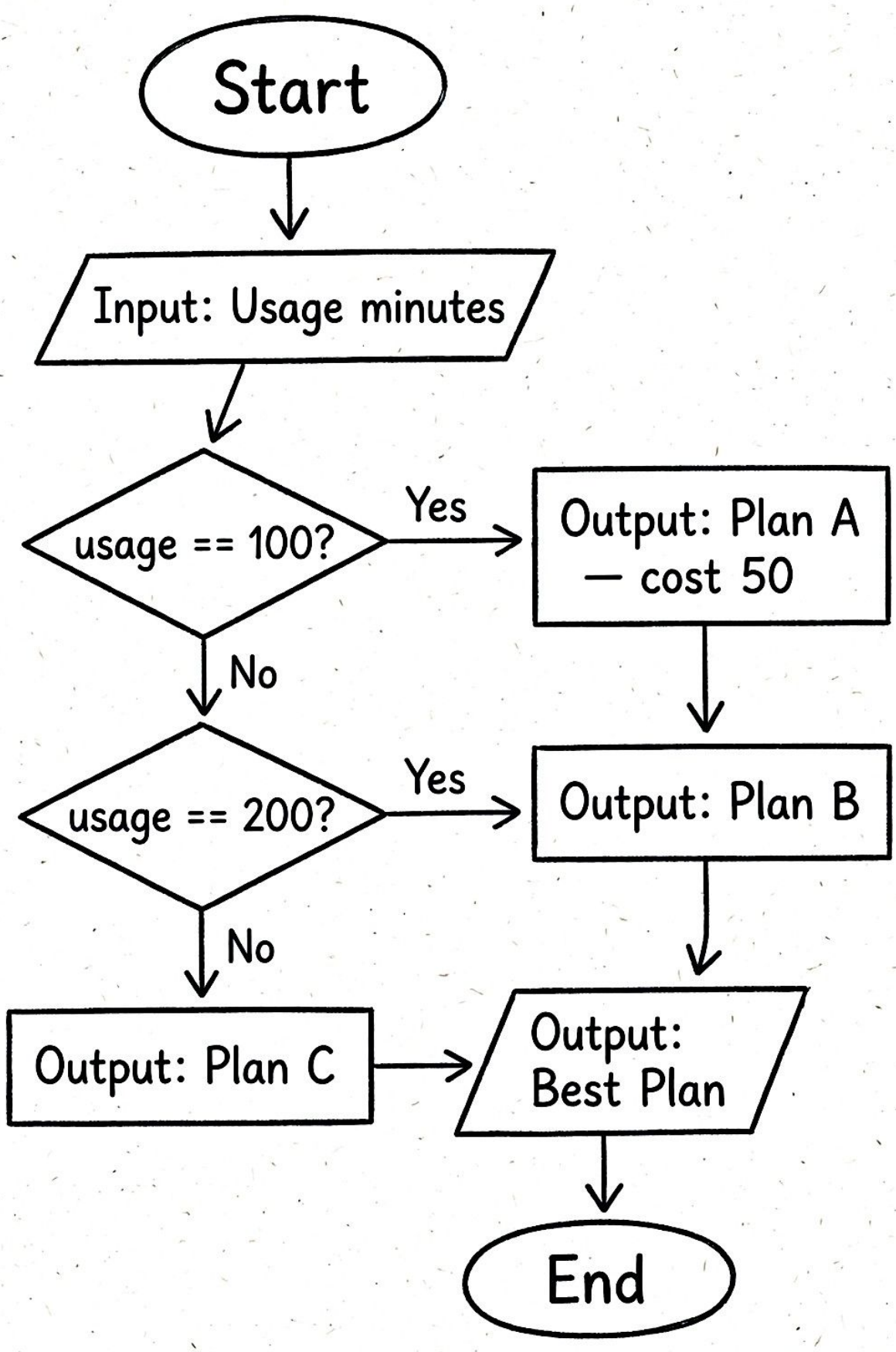
- Test Case 1: 5 * 800 = 4000 → Discount: 0, GST: 0 → Final: 4000
- Test Case 2: 5 * 1000 = 5000 → Discount: 0, GST: 0 → Final: 5000
- Test Case 3: 10 * 600 = 6000 → Discount: 600, GST: 0 → Final: 5400
- Test Case 4: 10 * 600 = 6000 (Electronics) → Discount: 600, GST: 972 → Final: 6372

Output:
As shown above.

Flowchart: See next page.

— End of Case Study 7 —

Multi-Plan Telecom Cost Optimization



Case Study 8: Manufacturing Cost and Profitability Analysis

Format: Same as Case 1

Objective: To find profit or loss.

Assumption: cost = material + labour

Problem Statement: Check profitability.

Algorithm:

Step 1: Start
Step 2: Input material, labour; cost = material + labour
Step 3: Input selling
Step 4: If selling > cost then profit = selling - cost else loss = cost - selling
Step 5: Print
Step 6: End.

Pseudocode:

```
BEGIN
READ material, labour, selling
cost = material + labour
IF selling > cost THEN
profit = selling - cost
PRINT "Profit", profit
ELSE
loss = cost - selling
PRINT "Loss", loss
END.
```

Source Code:

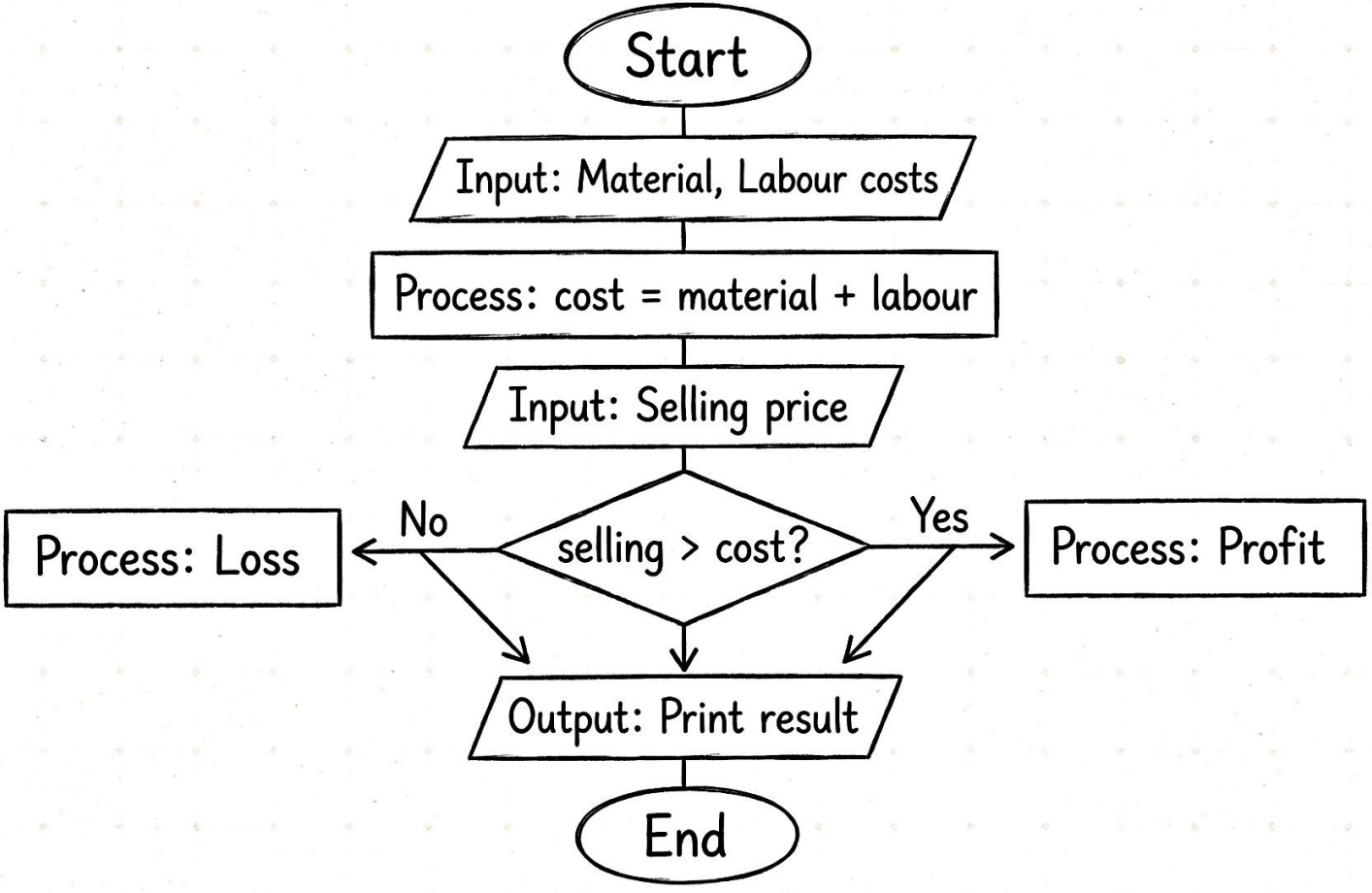
```
material = int(input("Material: "))
labour = int(input("Labour: "))
cost = material + labour
selling = int(input("Selling: "))
if selling > cost:
  print("Profit", selling - cost)
  else:
  print("Loss", cost - selling)
```

Test Cases:

- 1000 5000 2000 → Profit 500
- 1000 1000 1500 → Loss 500

--- End of Case Study 8 ---

Manufacturing Cost & Profitability



Case Study 9: Fuel Consumption and Route Comparison / Travel Budget

Format same as Case 1

Objective: To convert travel budget.

Assumption:

- 1 Dollar = 83 INR

Problem Statement: Convert dollars to INR & check budget.

Algorithm:

Step 1: Start
Step 2: Input dollars; rate = 83; inr = dollars * rate
Step 3: Input budget
Step 4: If inr > budget → Output "Over budget" else → Output "Within budget"
Step 5: End.

Pseudocode:

```
BEGIN
READ dollars
rate = 83
inr = dollars * rate
READ budget
IF inr > budget THEN
PRINT "Over budget"
PRINT "Within budget"
END.
```

Source Code:

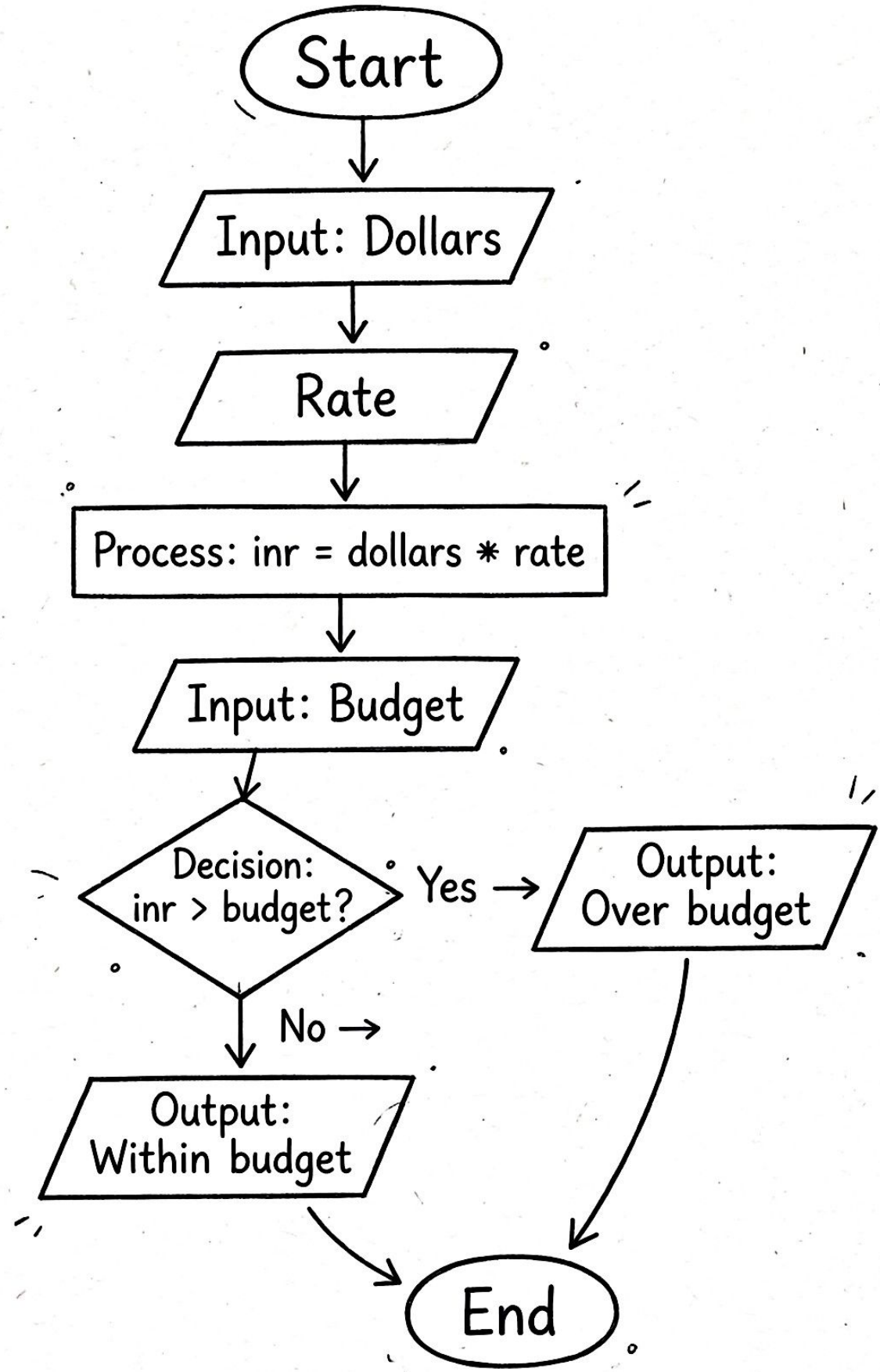
```
dollars = int(input("Dollars: "))
rate = 83
inr = dollars * rate
print("INR", inr)
budget = int(input("Budget: "))
if inr > budget:
  print("Over budget")
  else:
  print("Within budget")
```

Test Cases:

	Dollars	INR	Budget	Output
1.	30	2490	5000	Within
2.	100	8300	5000	Over

Note: All calculations are based on the assumption 1 Dollar = 83 INR.

Multi-Currency Travel Budget



Case Study 10: Campus Event Financial Planning

Format: Same as Case 1

Objective:
To check event profit.

Fonumption: Total = Venue + Food cost • Collection = Students × 100 (ticket price at 100 per student)

Problem Statement: Check whether the campus event results in a Profit or a Loss.

Algorithm:

1. Start
2. Input venue, food → compute total = venue + food
3. Input students → compute collection = students × 100
4. If collection ≥ total then print Profit else print Loss
5. End

Pseudocode:

```
BEGID venue, food, students
total = venue + food
collection = students * 100
IF collection >= total THEN
PRINT "Profit", (collection - total)
ELSE  "Loss", (total - collection)
END IF
```

Source Code: (Python)

```
venue = int(ihput("Venue: ")
food = int(ibput("Food: ")
total = venue + food
students = int(ihput("Students: ")
collection = students * 100
if collection >= total;
print("Profit", collection - total)
else:("Loss",  total - collection)
```

Test Cases:

Test Case	Input (venue, food, students)	Expected Output
1.	1000, 2000, 40	Profit 1000
2.	2000, 3000, 50	Break-even 0
3.	5000, 5000, 50	Loss 5000

Note: Profit is calculated as collection - total. Loss is calculated as total - collection. Break-even occurs when collection equals total.

Campus Event Financial Planning

